

Appendix for “Causal Newton Optimization: Online Calibration with Iterative Local Linear Modeling and Newton Updates”

Daigo Fujiwara^{1,3}, Tomonori Izumitani¹ and Shohei Shimizu^{2,3}

¹NTT DOCOMO BUSINESS, Inc.

²The University of Osaka

³Shiga University

{d.fujiwara, tomonori.izumitani}@ntt.com, shimizu.shohei@gmail.com

A Newton’s Method and Secant Method

Theorem 1 (Newton’s Method, Secant Method).

Newton’s method: Consider the optimization problem of a convex function $G(c) : \mathbb{R} \rightarrow \mathbb{R}$, which is equivalent to the root-finding problem of $g(c) := G'(c) = 0$. Assume there exists a closed interval I containing the optimal value c^* , such that $g(c)$ is of class C^2 and $g'(c) \neq 0$ in I . Given an initial value c_0 sufficiently close to c^* , the update sequence defined by:

$$c_{m+1} = c_m - \frac{g(c_m)}{g'(c_m)} \quad (1)$$

converges quadratically to c^* .

Secant method: When using another update equation where $g'(c_m)$ is replaced with a difference quotient,

$$c_{m+1} = c_m - \frac{(c_m - c_{m-1})g(c_m)}{g(c_m) - g(c_{m-1})}, \quad (2)$$

c_m converges to c^* in the $\frac{1+\sqrt{5}}{2}$ -th order.

B Proof for Local Convexity of Objective Function

Proof. Assume that the objective function $G(c)$ is twice continuously differentiable (C^2) and that c^* is a local minimum of $G(c)$. We also assume that c^* is an interior point of the domain of G .

1. Stationary Point: By Fermat’s theorem (also known as interior extremum theorem), since c^* is a local minimum and an interior point, it must be a stationary point. This means the gradient of G at c^* is zero:

$$G'(c^*) = 0.$$

2. Taylor Expansion: Expand $G(c)$ around c^* using Taylor’s theorem (up to the second-order term):

$$\begin{aligned} G(c) &= G(c^*) + G'(c^*)(c - c^*) \\ &\quad + \frac{1}{2}G''(c^*)(c - c^*)^2 + O(|c - c^*|^3) \\ &= G(c^*) + \frac{1}{2}G''(c^*)(c - c^*)^2 + O(|c - c^*|^3), \end{aligned}$$

where $G'' (= g'(c))$ is the Hessian of G evaluated at c^* , and $O(|c - c^*|^3)$ represents higher-order terms that become negligible as c approaches c^* .

3. Local Minimum: Since c^* is a local minimum, for all c sufficiently close to c^* , we have:

$$G(c) \geq G(c^*).$$

4. Hessian: Combining all, we get:

$$\frac{1}{2}G''(c^*)(c - c^*)^2 + O(|c - c^*|^3) \geq 0. \quad (3)$$

5. Local Convexity: A twice continuously differentiable function is locally convex in a neighborhood of a point if its Hessian is positive semi-definite, i.e. $G''(c^*) \geq 0$, at that point. When c is sufficiently close to c^* , $O(|c - c^*|^3)$ is ignorable compared to $\frac{1}{2}G''(c^*)(c - c^*)^2$, so $G''(c^*) \geq 0$ is a necessary condition of (3). Therefore, $G(c)$ is locally convex in a neighborhood of c^* . □

C Back-door Criterion

Definition 1 (Back-door criterion). Given an ordered pair of variables (X, Y) in causal graph $\mathcal{G}$, a set of endogenous variables $\mathbf{Z}$ satisfies the back-door criterion if no element of $\mathbf{Z}$ is a descendent of X , and $\mathbf{Z}$ can block all paths directed to X in all paths connecting X and Y .

D Proof for Lemma 4

For the clarity of the proofs in this section, we introduce a dedicated notation for the input vector to the function f_i :

$$\xi_i := (\text{pa}_i, U_i)^T.$$

We also define its conditional expectation as $\bar{\xi}_i := \mathbb{E}[\xi_i | \text{do}(X = c_m)]$.

Lemma 1 (Local Linear Approximation). Let $H_i = \nabla^2 f_i(\bar{\xi}_i)$ be the Hessian of f_i at $\bar{\xi}_i$, and let $\lambda_{\max}^{(i)}$ be its largest eigenvalue in absolute value. If the deviation $\|\xi_i - \bar{\xi}_i\|$ is sufficiently small to satisfy $\|\xi_i - \bar{\xi}_i\| \ll \frac{\|\nabla f_i(\bar{\xi}_i)\|}{|\lambda_{\max}^{(i)}|}$, then f_i can be approximated around $\bar{\xi}_i$ as:

$$f_i(\xi_i) \simeq f_i(\bar{\xi}_i) + \nabla f_i(\bar{\xi}_i)^T(\xi_i - \bar{\xi}_i). \quad (4)$$

Proof of Lemma 1. The Taylor expansion of $f_i(\xi_i)$ around $\bar{\xi}_i$ is given by:

$$\begin{aligned} f_i(\xi_i) &= f_i(\bar{\xi}_i) + \underbrace{\nabla f_i(\bar{\xi}_i)^T (\xi_i - \bar{\xi}_i)}_{\text{1st-order term}} \\ &\quad + \underbrace{\frac{1}{2}(\xi_i - \bar{\xi}_i)^T H_i(\xi_i - \bar{\xi}_i)}_{\text{2nd-order term}} + O(\|\xi_i - \bar{\xi}_i\|^3). \end{aligned}$$

A linear approximation is valid when the second-order term is negligible compared to the first-order term. We compare the magnitudes of these two terms.

The magnitude of the first-order term can be bounded using the Cauchy-Schwarz inequality:

$$|\nabla f_i(\bar{\xi}_i)^T (\xi_i - \bar{\xi}_i)| \leq \|\nabla f_i(\bar{\xi}_i)\| \cdot \|\xi_i - \bar{\xi}_i\|. \quad (5)$$

The magnitude of the second-order term is bounded using a property of quadratic forms (specifically, the Rayleigh quotient):

$$\left| \frac{1}{2}(\xi_i - \bar{\xi}_i)^T H_i(\xi_i - \bar{\xi}_i) \right| \leq \frac{1}{2} |\lambda_{\max}^{(i)}| \cdot \|\xi_i - \bar{\xi}_i\|^2, \quad (6)$$

where $|\lambda_{\max}^{(i)}|$ is the largest absolute eigenvalue of the Hessian H_i .

For the linear approximation to hold, we require the magnitude of the second-order term to be significantly smaller than that of the first-order term:

$$\frac{1}{2} |\lambda_{\max}^{(i)}| \cdot \|\xi_i - \bar{\xi}_i\|^2 \ll \|\nabla f_i(\bar{\xi}_i)\| \cdot \|\xi_i - \bar{\xi}_i\|.$$

Assuming $\|\xi_i - \bar{\xi}_i\| \neq 0$, we can divide both sides by this term, which leads to the condition stated in the lemma:

$$\|\xi_i - \bar{\xi}_i\| \ll \frac{2\|\nabla f_i(\bar{\xi}_i)\|}{|\lambda_{\max}^{(i)}|}. \quad \square$$

(The constant factor 2 is often omitted in such "much smaller than" relations, as it does not change the core condition.)

And then inversely, the linear approximation of $V_i = f_i(\xi_i)$ holds:

$$V_i = f_i(\xi_i) \simeq f_i(\bar{\xi}_i) + \nabla f_i(\bar{\xi}_i)^T (\xi_i - \bar{\xi}_i).$$

$\square$

Proof of Lemma 4. For the entire Structural Causal Model (SCM) to be locally approximated by a linear model, the condition of Lemma 1 must hold recursively for all exogenous variables $i = 1, \beta^T Z, P$. Think about what conditions can we expect this to be true.

The key is to ensure that the deviation of a variable from its mean does not get amplified as it propagates through the structural equations (graph). Consider an endogenous variable V_i . From the linear approximation in Lemma 1, the deviation of $V_i = f_i(\xi_i)$ from its mean is primarily driven by the first-order term. Its magnitude is proportional to $\|\nabla f_i(\bar{\xi}_i)\|$ and the deviation of its parents, $\|\xi_i - \bar{\xi}_i\|$.

If the gradient norm $\|\nabla f_i(\bar{\xi}_i)\|$ is large (i.e., greater than 1), it can amplify the deviation from the parents. This amplified deviation then becomes part of the input deviation for

its children variables, potentially causing their linear approximations to fail. This can create a cascading effect of approximation failures down the structural equations.

Therefore, for the entire SCM to be robustly linearizable, a sufficient condition is that the gradients at all structural equations are non-amplifying, i.e., $\|\nabla f_j(\bar{\xi}_j)\|$ is **sufficiently small (e.g., ≤ 1) for all nodes j** . If this condition holds, deviations will not grow uncontrollably through the graph, ensuring that the local neighborhood condition $\|\xi_i - \bar{\xi}_i\|$ remains small enough for all nodes i to satisfy the premise of Lemma 1. $\square$

Note that, in order to achieve both diametrically opposed requirements of $\|\xi_i - \bar{\xi}_i\| \ll \frac{\|\nabla f_i(\bar{\xi}_i)\|}{|\lambda_{\max}^{(i)}|}$ and sufficiently small $\|\nabla f_i(\bar{\xi}_i)\|$, the Hessian scale $|\lambda_{\max}^{(i)}|$ must be sufficiently small. This is equivalent to requiring that the second-order derivative, which represents the degree of nonlinearity ($\simeq$ the degree of function curvature), be small in the neighborhood, which is also intuitively correct.

E Derivatives under Log-Linear Relationship

Lemma 2 (Derivatives under Local Log-Linear Relationship). Assume that a function $G_2(c) > 0$ can be locally modeled by a log-linear function with respect to c , i.e., $\ln G_2(c) \simeq w_0 + w_c c \Leftrightarrow G_2(c) \simeq \exp(w_0 + w_c c)$. Then, the first and second derivatives of the function, $G'_2(c) = g_2(c)$ and $G''_2(c) = g'_2(c)$, satisfy the following relationships:

$$g_2(c) = w_c G_2(c), \quad g'_2(c) = w_c^2 G_2(c). \quad (7)$$

F Statistical Properties of the Squared Bias Estimator

In this section, we analyze the statistical properties of the estimators used for the squared bias term of the objective function, $G_1(c) = (\mu(c) - Y^*)^2$. Specifically, we show that while the plug-in estimator for the objective value is biased, the estimator for its gradient—which determines the Newton update direction—is unbiased.

Proposition 1 (Bias of the Objective Value Estimator). Let $\hat{\mu}_m$ be the sample mean of the target variable Y obtained from T samples under the intervention c_m . The plug-in estimator for the squared bias, defined as $\hat{G}_1(c_m) := (\hat{\mu}_m - Y^*)^2$, is a biased estimator of the true squared bias $G_1(c_m)$.

Proof. Let $\mu(c_m) = \mathbb{E}[Y \mid \text{do}(X = c_m)]$ be the true expected value. The sample mean $\hat{\mu}_m$ is an unbiased estimator of $\mu(c_m)$ with variance $\text{Var}(\hat{\mu}_m) = \sigma^2/T$, where σ^2 is the variance of Y . We can write $\hat{\mu}_m = \mu(c_m) + \delta$, where $\mathbb{E}[\delta] = 0$ and $\mathbb{E}[\delta^2] = \text{Var}(\hat{\mu}_m)$.

Taking the expectation of the estimator $\hat{G}_1(c_m)$:

$$\begin{aligned} \mathbb{E}[\hat{G}_1(c_m)] &= \mathbb{E}[(\hat{\mu}_m - Y^*)^2] \\ &= \mathbb{E}[(\mu(c_m) - Y^* + \delta)^2] \\ &= \mathbb{E}[(\mu(c_m) - Y^*)^2 + 2(\mu(c_m) - Y^*)\delta + \delta^2] \\ &= (\mu(c_m) - Y^*)^2 + 2(\mu(c_m) - Y^*) \underbrace{\mathbb{E}[\delta]}_0 + \underbrace{\mathbb{E}[\delta^2]}_{\text{Var}(\hat{\mu}_m)} \\ &= G_1(c_m) + \text{Var}(\hat{\mu}_m). \end{aligned} \quad (8)$$

Thus, $\hat{G}_1(c_m)$ overestimates the true value by the variance of the sample mean. $\square$

Proposition 2 (Unbiasedness of the Gradient Estimator). *Despite the bias in the objective value, the estimator for the gradient of the squared bias term used in the Newton update, $\hat{g}_1(c_m) := 2(\hat{\mu}_m - Y^*)\hat{\mu}'_m$, is an unbiased estimator of the true gradient $g_1(c_m)$, assuming the local causal effect estimator $\hat{\mu}'_m$ is obtained via Ordinary Least Squares (OLS) and errors are normally distributed.*

Proof. The true gradient is given by $g_1(c_m) = 2(\mu(c_m) - Y^*)\mu'(c_m)$. In standard linear regression analysis, the estimator for the mean at the centroid of the data (or the intercept in a centered model), $\hat{\mu}_m$, and the estimator for the causal effect, $\hat{\mu}'_m$ (denoted as $\hat{w}_{YX}$ in the main text), are statistically independent (or at least uncorrelated).

Using this independence property:

$$\begin{aligned}\mathbb{E}[\hat{g}_1(c_m)] &= \mathbb{E}[2(\hat{\mu}_m - Y^*)\hat{\mu}'_m] \\ &= 2\mathbb{E}[\hat{\mu}_m - Y^*] \cdot \mathbb{E}[\hat{\mu}'_m] \\ &= 2(\mathbb{E}[\hat{\mu}_m] - Y^*) \cdot \mathbb{E}[\hat{\mu}'_m] \\ &= 2(\mu(c_m) - Y^*) \cdot \mu'(c_m) \\ &= g_1(c_m).\end{aligned}\quad (9)$$

Here, we utilized the unbiasedness of the individual estimators: $\mathbb{E}[\hat{\mu}_m] = \mu(c_m)$ and $\mathbb{E}[\hat{\mu}'_m] = \mu'(c_m)$. $\square$

Remark. The result of Proposition 2 ensures that the optimization process of CNO is driven by a correct descent direction on average, even though the estimated value of the objective function itself contains a positive bias as shown in Proposition 1. This justifies the use of $\hat{\mu}_m$ for the gradient calculation $\hat{g}_1$.

Proposition 3 (Bias of the Second-order Derivative Estimator and Damping Effect). *The estimator for the approximated second-order derivative of the squared bias term, $\hat{g}'_1(c_m) := 2(\hat{\mu}'_m)^2$, is a positively biased estimator of the true Gauss-Newton curvature $g'_1(c_m) \approx 2(\mu'(c_m))^2$.*

Proof. Let $\sigma_w^2 = \text{Var}(\hat{\mu}'_m)$ be the variance of the local causal effect estimator derived from OLS. Using the definition of variance, $\mathbb{E}[X^2] = (\mathbb{E}[X])^2 + \text{Var}(X)$, we have:

$$\begin{aligned}\mathbb{E}[\hat{g}'_1(c_m)] &= 2\mathbb{E}[(\hat{\mu}'_m)^2] \\ &= 2((\mathbb{E}[\hat{\mu}'_m])^2 + \text{Var}(\hat{\mu}'_m)) \\ &= 2(\mu'(c_m))^2 + 2\sigma_w^2 \\ &= g'_1(c_m) + 2\sigma_w^2.\end{aligned}\quad (10)$$

Since variances are non-negative, $\mathbb{E}[\hat{g}'_1(c_m)] \geq g'_1(c_m)$, indicating a positive bias. $\square$

Remark. Although biased, this overestimation of the curvature (denominator in the Newton update) is beneficial in stochastic settings. It acts as an implicit regularization or damping factor: when the uncertainty in the causal effect estimation (σ_w^2) is high, the denominator increases, naturally reducing the step size and preventing unstable updates driven by noisy gradient estimates.

Proposition 4 (Bias in Variance Estimation using Sample Mean Residuals). *Let $r_{\text{naive}}^{(t)} := y^{(t)} - \hat{\mu}_m$ be the residual computed using the scalar sample mean $\hat{\mu}_m$. The variance estimator derived from these residuals, $\hat{G}_{2,\text{naive}}(c_m) := \frac{1}{T} \sum_t (r_{\text{naive}}^{(t)})^2$, is a positively biased estimator of the true system variance $G_2(c_m) = \text{Var}(\epsilon^{(t)})$. Specifically, it overestimates the variance by a term proportional to the squared local causal effect and the variance of the intervention noise.*

Proof. Consider the local linear approximation of the system dynamics around the intervention c_m :

$$y^{(t)} = \mu(c_m) + w_{YX}(x^{(t)} - c_m) + \epsilon^{(t)}, \quad (11)$$

where $x^{(t)} = c_m + e^{(t)}$ with intervention noise $e^{(t)} \sim (0, \sigma_e^2)$, and $\epsilon^{(t)} \sim (0, \sigma_\epsilon^2)$ represents the intrinsic system noise (the target variance G_2).

Substituting the sample mean $\hat{\mu}_m \approx \mu(c_m)$ (ignoring the estimation error of the mean for simplicity as it is of lower order compared to the trend), the naive residual is:

$$\begin{aligned}r_{\text{naive}}^{(t)} &\approx (\mu(c_m) + w_{YX}e^{(t)} + \epsilon^{(t)}) - \mu(c_m) \\ &= w_{YX}e^{(t)} + \epsilon^{(t)}.\end{aligned}\quad (12)$$

Taking the expectation of the squared residual:

$$\begin{aligned}\mathbb{E}[(r_{\text{naive}}^{(t)})^2] &\approx \mathbb{E}[(w_{YX}e^{(t)} + \epsilon^{(t)})^2] \\ &= w_{YX}^2 \mathbb{E}[(e^{(t)})^2] + \mathbb{E}[(\epsilon^{(t)})^2] + 2w_{YX} \underbrace{\mathbb{E}[e^{(t)}\epsilon^{(t)}]}_0 \\ &= w_{YX}^2 \sigma_e^2 + \sigma_\epsilon^2.\end{aligned}\quad (13)$$

Thus, the estimator converges to $G_2(c_m) + w_{YX}^2 \sigma_e^2$, which includes an artifact term dependent on the local causal effect w_{YX} and the exploration noise magnitude. $\square$

Corollary 1 (Necessity of Regression-Based Residuals). *To obtain a consistent estimator of the intrinsic variance $G_2(c_m)$ and its gradient, one must use regression-based residuals $r^{(t)} = y^{(t)} - \hat{\mu}(x^{(t)}, z^{(t)})$. By using the local linear predictor $\hat{\mu}(x^{(t)}, z^{(t)})$ instead of the scalar sample mean, the trend component $w_{YX}e^{(t)}$ is absorbed by the regression model, leaving $r^{(t)} \approx \epsilon^{(t)}$, thereby isolating the true heteroscedasticity of the system.*

G Distribution (Variance) Adjustment

If a set of variables Z satisfies the back-door criterion relative to (X, Y) , it blocks all spurious paths from X to Y , thereby identifying the interventional distribution $P(Y \mid do(X), Z) = P(Y \mid X, Z)$. Since the conditional variance is a functional of this distribution, specifically $E[Y^2 \mid do(X)] - (E[Y \mid do(X)])^2$, the identifiability provided by the back-door adjustment extends to the variance [Pearl, 2009; Pearl *et al.*, 2016]. Therefore, including Z in Equation (12) correctly adjusts for confounding biases in the variance estimation, identifying the causal effect of X on the spread of Y .

Algorithm 1 Causal Newton Optimization (CNO)

Inputs: Causal graph $\mathcal{G}$, target value Y^* , initial data D_{init} , number of iterations M , intervention interval T .

```
1:  $D, c \leftarrow D_{\text{init}}, 0$ 
2: for  $m \leftarrow 1, M$  do
3:   # 1. Estimate local statistics from data  $D$ 
4:    $\hat{\mu} \leftarrow$  Sample mean of target variable  $Y$ .
5:    $\hat{\mu}' \leftarrow$  Estimate causal effect by OLS .
6:    $\hat{G}_2 \leftarrow$  Estimate variance of  $Y$  with residual.
7:    $\hat{\omega}_{YX} \leftarrow$  Estimate coefficient by log-residual OLS.
8:   # 2. Approximate derivatives of  $G(c)$ 
9:    $\hat{g}_1, \hat{g}_1' \leftarrow$  calculate (11) with  $\hat{\mu}, \hat{\mu}'$ 
10:   $\hat{g}_2, \hat{g}_2' \leftarrow$  calculate (13) with  $\hat{G}_2, \hat{\omega}_{YX}$ 
11:  # 3. Update correction term  $c$  via Newton's method
12:   $c \leftarrow c - (\hat{g}_1 + \hat{g}_2)/(\hat{g}_1' + \hat{g}_2')$ 
13:  # 4. Collect new data for the next iteration
14:   $D \leftarrow \emptyset$ 
15:  for  $t \leftarrow 1, T$  do
16:     $D \leftarrow D \cup$  data via the new intervention  $c$ .
17:  end for
18: end for
```

H Coefficient Unbiasedness under Log-Transformation

Proposition 5 (Coefficient Unbiasedness under Log-Transformation). *Assume a multiplicative noise model $r = \sqrt{V(\zeta)}\epsilon$, where the noise ϵ is independent of ζ . Although the log-transformation introduces a bias in the intercept estimation due to Jensen's inequality, the Ordinary Least Squares (OLS) estimator for the coefficient ω remains an unbiased estimator of the derivative of the log-variance with respect to ζ .*

Proof. Taking the logarithm of the squared residuals yields $\ln(r^2) = \ln V(\zeta) + \ln(\epsilon^2)$. The conditional expectation is $E[\ln(r^2) \mid \zeta] = \ln V(\zeta) + E[\ln(\epsilon^2)]$. While the term $E[\ln(\epsilon^2)]$ represents a bias (differing from $\ln E[\epsilon^2]$), it is constant with respect to ζ due to the independence of ϵ . Consequently, this bias affects only the intercept, and the coefficient of the regression function consistently captures $\frac{d}{d\zeta} \ln V(\zeta)$. $\square$

I CNO Algorithm

See Algorithm 1.

J Causal Newton Optimization (CNO) Variants

We consider several variants of the Causal Newton Optimization (CNO) algorithm. These variants differ primarily in how they handle the variance term $G_2(c)$ of the objective function and the type of intervention used for exploration.

J.1 Ensuring Variance for Regression under Additive Interventions

The correction term c_m calculated by our method is a single, deterministic value at each iteration. However, estimat-

ing the local causal effect via regression requires the manipulated variable X to possess non-zero variance.

Because our proposed CNO algorithm fundamentally rests on additive interventions ($\text{do}(X \leftarrow X_{\text{nat}} + c)$) rather than standard hard interventions, it inherently addresses this variance requirement in a safe and non-disruptive manner. Under an additive intervention, we can directly leverage the natural fluctuations of the unrealized natural value X_{nat} generated by the underlying system dynamics. This approach is highly consistent with our calibration setting; by preserving the system's inherent feedback mechanisms and utilizing its already semi-optimized natural variance, it represents a far safer exploration strategy than hard interventions.

Nevertheless, depending on the magnitude of the system's natural fluctuations, relying solely on the inherent variance of X_{nat} might sometimes yield insufficient information for highly accurate regression. To ensure robust system identification, we consider two implementation strategies within our additive intervention framework to manage the explanatory variance:

- **AddNatural:** Relying exclusively on the natural variations of X_{nat} to provide the required variance for regression, injecting no artificial disturbances to ensure maximum safety.
- **AddExcitation:** Intentionally superimposing a minor exploratory random noise $e^{(t)}$ —well-known as an excitation signal in conventional system identification [Ljung, 1998]—onto the correction term, resulting in $\text{do}(X^{(t)} \leftarrow X_{\text{nat}}^{(t)} + c_m + e^{(t)})$.

While the additive intervention alone ensures a baseline variance without disruptive shifts, our numerical experiments evaluate and compare both the purely natural variation approach (**AddNatural**) and the noise-augmented approach (**AddExcitation**). Based on empirical performance, the combination of additive intervention and excitation noise (**AddExcitation**) is ultimately adopted in our final algorithm to guarantee sufficient exploration and regression stability, as formalized in Algorithm 1.

J.2 The Problem of Variance Overestimation

As formally proven in Appendix F (Proposition 4), estimating the variance of the target variable using simple sample variance leads to systematic overestimation. This bias, proportional to the squared local causal effect and the variance of the exploration noise ($w_{YX}^2 \mathbb{V}[e^{(t)}]$), is particularly problematic when the local causal effect w_{YX} is large. Therefore, the proposed CNO utilizes regression-based residuals to isolate true system heteroscedasticity.

J.3 CNO Variants: Details

- **CNO (BiasOnly):** This variant serves as a baseline, optimizing only the squared bias term $G_1(c)$ of the objective function. It represents a simpler approach that ignores the effect of the intervention on the target's variance, effectively assuming $G_2(c)$ is constant.
- **CNO (Unadjusted):** This variant does not perform the back-door adjustment, meaning it ignores the presence

of confounders $z^{(t)}$ during the local causal effect estimation. Consequently, it is highly susceptible to confounding bias, which can drive the optimization in the completely wrong direction in feedback control settings.

- **CNO (Hard):** Unlike the proposed method that uses additive corrections, this variant employs standard hard interventions ($\text{do}(X = a)$) to fix the input to a constant value. While it breaks the causal links from parents to X , it requires artificial excitation noise to successfully identify the causal effect and lacks the safety benefits of preserving the system’s natural feedback loops.
- **CNO (SampleVar):** This method estimates the variance term $G_2(c)$ using the raw Sample Variance (SampleVar) of the observed outcomes $y^{(t)}$ for a given intervention value c_m . The estimator is given by:

$$\hat{G}_{2,\text{SV}}(c_m) = \frac{1}{T-1} \sum_{t=mT+1}^{(m+1)T} (y^{(t)} - \hat{\mu}(c_m))^2.$$

While simple to implement, this approach systematically overestimates the true system variance because the observed variance includes variance induced by the exploration noise that is intentionally added to the intervention value.

- **CNO (ResVar):** This method aims to provide a less biased estimate of $G_2(c)$ by using the Residual Variance (RV) from the local linear causal model. It calculates the variance of the residuals from the regression used to estimate the causal effect. The estimator is given by:

$$\hat{G}_{2,\text{RV}}(c_m) = \frac{1}{T-2} \sum_{t=mT+1}^{(m+1)T} (y^{(t)} - \hat{w}_0 - \hat{w}_{YX} x^{(t)})^2$$

where $\hat{w}_0$ and $\hat{w}_{YX}$ are the estimated coefficients from the regression of $y^{(t)}$ on $x^{(t)}$ and $z^{(t)}$. This method corrects for the bias introduced by the exploration noise by focusing on the variance of the residuals, but it can still be sensitive to model misspecification. If the local linear model is a poor fit for a highly non-linear system, the variance estimate can be inaccurate.

- **CNO (NoiseSubVar):** This method provides an alternative bias correction by starting with the Sample Variance and explicitly subtracting the estimated variance component attributable to the known exploration noise. The estimator is given by:

$$\hat{G}_{2,\text{NS}} = \hat{G}_{2,\text{SV}} - \hat{w}_{YX}^2 \mathbb{V}[e].$$

This Noise Subtraction (NS) approach is computationally efficient but is also sensitive to errors in the estimated causal effect ($\hat{w}_{YX}$) and, more critically, does not guarantee a non-negative variance estimate, which can lead to numerical instability. In practical terms, if a negative value is obtained, it is replaced with 0.

- **CNO (AddNatural) & CNO (AddExcitation):** These variants employ additive intervention strategies instead of hard interventions. **CNO (AddNatural)** leverages the

natural fluctuations of the manipulated variable for system identification. **CNO (AddExcitation)** builds on this by additionally injecting a known excitation noise to ensure sufficient exploration. In **CNO (AddExcitation)**, the variance is estimated using $\hat{G}_{2,\text{NS}}$. Due to its superior empirical accuracy and robust performance demonstrated across our experiments, **CNO (AddExcitation)** is ultimately adopted as the final specification of our proposed CNO algorithm.

K Threshold Stopping Update of Derivatives of $G_2(c)$ (Deprecated)

If we estimate the derivatives of the variance term $G_2(c) = V[Y|\text{do}(X = a)]$ by finite differences, i.e. using the secant method, the update rules are given by:

$$\hat{g}_{2,m} \simeq \frac{\hat{G}_2(c_m) - \hat{G}_2(c_{m-1})}{c_m - c_{m-1}}$$

$$\hat{g}'_{2,m} \simeq \frac{\hat{g}_{2,m} - \hat{g}_{2,m-1}}{c_m - c_{m-1}}$$

A practical issue arises with this approach. As the optimization process converges, the intervention value c_m approaches the optimal value c^* . Consequently, the difference between consecutive intervention values, $|c_m - c_{m-1}|$, becomes very small. This can lead to numerical instability in the derivative estimations due to division by a near-zero value, potentially amplifying the estimation errors in $\hat{G}_2(c)$.

To address this, we introduce a practical mechanism to halt the updates of these derivatives when the change in the intervention value is sufficiently small. This is implemented in the ‘optimize’ method within the ‘CausalNewtonOptimization’ class in our Python code.

The core of this mechanism is the following conditional block:

```
if m > 0 and np.abs(c - c_tilde) > self._delta:
    self.count_threshold_clear += 1
    g2 = (G2 - G2_tilde) / (c - c_tilde)
    g2_prime = (g2 - g2_tilde) / (c - c_tilde)
```

Here, ‘c’ corresponds to the current intervention value c_m , and ‘c_tilde’ holds the value from the previous step, c_{m-1} . The derivatives ‘g2’ ($\hat{g}_{2,m}$) and ‘g2_prime’ ($\hat{g}'_{2,m}$) are updated only if the absolute difference $|c_m - c_{m-1}|$ exceeds a predefined threshold, ‘_delta’ (δ). If the difference does not exceed the threshold, the values of ‘g2’ and ‘g2_prime’ from the previous iteration are retained, effectively stopping the update.

The threshold δ can be configured to be either a fixed value or determined dynamically. In the “Auto” mode, it is calculated at each step m as:

$$\delta = \alpha \cdot \frac{\hat{G}_2(c_m)}{\sqrt{T}}$$

where ‘alpha’ (α) is a hyperparameter, $\hat{G}_2(c_m)$ is the estimated variance at the current step, and T is the number of data points collected per intervention. This dynamic adjustment allows the threshold to scale with the magnitude of the

system’s variance, providing a more robust criterion for stopping the derivative updates.

This thresholding mechanism is a crucial heuristic that ensures the stability and reliable convergence of the CNO algorithm, particularly in the later stages of optimization where fine-tuning of the intervention value is required.

L A Brief Overview of Soft Actor-Critic (SAC)

Soft Actor-Critic (SAC) is an off-policy actor-critic deep reinforcement learning algorithm based on the maximum entropy reinforcement learning framework [Haarnoja *et al.*, 2018]. The primary goal of SAC is to maximize a trade-off between the expected return and the policy’s entropy. By encouraging higher entropy, the policy is incentivized to explore more broadly and avoid premature convergence to a suboptimal deterministic policy, which contributes to its stability and performance.

L.1 Implementation Details

The SAC algorithm is implemented in the file `mymodules/optimizers/soft_actor_critic.py` using the `stable_baselines3` and `gymnasium` libraries. To apply SAC to our online optimization problem, which is defined by a Structural Causal Model (SCM) rather than a traditional Markov Decision Process (MDP), we map our problem’s components to the standard reinforcement learning framework. This is achieved by creating a custom `gymnasium` environment that serves as a wrapper around our data generation process. The key components are mapped as follows:

- **Environment:** The target system itself, which is described by the SCM. At each step, it receives an intervention and generates a new batch of data.
- **Action (c_m):** The action corresponds to the intervention value c_m that the agent chooses to apply to the manipulatable variable X . The action space is continuous, defined by a given `intervention_range`.
- **Reward (r_m):** The reward is designed to guide the agent towards minimizing our objective function $G(c) = E[(Y - Y^*)^2 | do(X = a)]$. It is defined as the negative of the Monte Carlo approximation of this objective, calculated from the data batch D_m collected over T time steps after applying action c_m :

$$r_m = -\frac{1}{T} \sum_{t=mT+1}^{(m+1)T} (y^{(t)} - Y^*)^2$$

- **State (s_m):** The state represents the information available to the agent for selecting the next action. Since the full data batch D_m can be high-dimensional, we also use a low-dimensional vector of its key statistics. In our main implementation, the state at step m (after applying intervention c_{m-1}) is a 3-dimensional vector composed of:

1. The previous intervention value, c_{m-1} .

2. The sample mean of the target variable, $\hat{\mu}(c_{m-1})$.
3. The sample variance of the target variable, $\hat{G}_2(c_{m-1})$.

For comparison purposes, we also test a variant (SAC (BatDataState)) where the state is a high-dimensional vector formed by concatenating the previous action c_{m-1} with the entire flattened data batch D_m . As a result, basically the method using all data D_m rather than statistical quantities proved to be more accurate, so we adopted this method for the experiments in this paper. Furthermore, in line with the discussion in the main text, both standard ‘do’-interventions and additive interventions are implemented and can be selected. Each optimization trial consists of M steps, where the agent interacts with the environment to learn an optimal policy.

Parameter Tuning

The performance of a SAC is highly dependent on its action range hyperparameters “(lower, upper)”. We hand tuned these parameters using the Linear SCM. As a result, the final action range parameters were set to lower = -3.0 and upper = 3.0. In Complex Synthetic Environment and CSTR Simulator experiments, the same parameters were used as a baseline for comparison with the CNO algorithm.

L.2 Algorithm Overview

Objective Function

The core objective of SAC is to learn a policy π that maximizes the entropy-augmented expected return. The temperature parameter, $\alpha > 0$, controls the stochasticity of the optimal policy by weighting the importance of the entropy term relative to the reward.

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))]$$

Here, s_t denotes the state, a_t the action, r the reward function, and ρ_π represents the state-action marginals of the trajectory distribution induced by the policy π . The term $\mathcal{H}(\pi(\cdot | s_t))$ is the entropy of the policy at state s_t .

Key Components

The SAC algorithm is implemented using several function approximators, typically neural networks:

- **Stochastic Policy (Actor)** $\pi_\phi(a_t | s_t)$: A network parameterized by ϕ that outputs a probability distribution over actions given a state.
- **Two Q-Networks (Critic)** $Q_{\theta_1}(s_t, a_t)$ and $Q_{\theta_2}(s_t, a_t)$: To mitigate positive bias in the value estimates, SAC employs the clipped double Q-learning technique from TD3 [Fujimoto *et al.*, 2018]. It learns two independent Q-functions, parameterized by θ_1 and θ_2 , and uses the minimum of their values for policy updates.
- **Two Target Q-Networks** $Q_{\theta_1, \text{target}}$ and $Q_{\theta_2, \text{target}}$: These are time-delayed copies of the critic networks, which are used to stabilize the training of the critics.

Learning Process

All networks are trained using mini-batches of transitions sampled from a replay buffer $\mathcal{D}$.

- **Critic Update:** The Q-networks are trained by minimizing the soft Bellman residual. The loss function for each critic Q_{θ_i} is:

$$L(\theta_i) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim \mathcal{D}} \left[(Q_{\theta_i}(s_t, a_t) - y(r_t, s_{t+1}))^2 \right]$$

where the target value y is computed using the minimum of the two target Q-networks and includes the entropy term:

$$\begin{aligned} y(r_t, s_{t+1}) &= r_t + \gamma \left(\min_{i=1,2} Q_{\theta_{i,\text{target}}}(s_{t+1}, a_{t+1}) - \alpha \log \pi_{\phi}(a_{t+1} | s_{t+1}) \right) \\ a_{t+1} &\sim \pi_{\phi}(\cdot | s_{t+1}) \end{aligned}$$

- **Actor Update:** The policy is updated by minimizing the following objective, which encourages actions that have high Q-values and high entropy:

$$J_{\pi}(\phi) = \mathbb{E}_{s_t \sim \mathcal{D}} \left[\mathbb{E}_{a_t \sim \pi_{\phi}} \left[\alpha \log \pi_{\phi}(a_t | s_t) - \min_{i=1,2} Q_{\theta_i}(s_t, a_t) \right] \right]$$

- **Automatic Temperature Tuning:** A key feature of SAC is the ability to automate the tuning of the temperature parameter α . This is framed as a constrained optimization problem where the objective is to maximize the expected return subject to a minimum entropy constraint. The loss function for α is:

$$J(\alpha) = \mathbb{E}_{a_t \sim \pi_{\phi}} [-\alpha \log \pi_{\phi}(a_t | s_t) - \alpha \bar{\mathcal{H}}]$$

where $\bar{\mathcal{H}}$ is a predefined target entropy, often set heuristically (e.g., as the negative of the action space dimension). This allows α to be adjusted dynamically during training.

M A Brief Overview of PID Control

The Proportional-Integral-Derivative (PID) controller is a classic feedback control algorithm widely used in industrial control systems and various other fields. In this study, a PID controller was employed as a baseline method for comparison with our proposed CNO algorithm. A PID controller works to minimize the error $e(t) = Y^* - y(t)$ between a desired setpoint Y^* and a measured process variable $y(t)$ by adjusting a control input $u(t)$. The control law is based on a linear combination of three terms:

- **Proportional Term:** Adjusts the control input in proportion to the current error $e(t)$. A larger error results in a larger control action.
- **Integral Term:** Addresses the accumulation of past errors by integrating them over time. This term is responsible for eliminating steady-state error.
- **Derivative Term:** Responds to the rate of change of the error. This term predicts future error and has a stabilizing effect, reducing overshoot and improving transient response.

M.1 Control Law

PID controllers are typically implemented in one of two forms: positional or velocity.

Positional Form

The standard textbook form of the PID controller calculates the absolute value of the control input $u(t)$ directly. The control law in continuous time is given by:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (14)$$

where K_p , K_i , and K_d are the proportional, integral, and derivative gains, respectively.

Velocity Form

In this work, we mainly use the velocity form, which offers several practical advantages such as easier anti-windup implementation and bumpless transfer between manual and automatic control. This form calculates the rate of change of the control input, $\frac{du(t)}{dt}$, which is then integrated to determine the current control action.

$$u_m = u_{m-1} + \Delta u_m \quad (15)$$

Here, u_m is the control input at discrete time step m , and Δu_m is the change in the control input for that step, calculated by a discretized PID law.

M.2 Parameter Tuning

The performance of a PID controller is highly dependent on its gain parameters (K_p , K_i , K_d). We hand tuned these parameters using the Linear SCM. As a result, the final gain parameters were set to $K_p = 0.1$, $K_i = 0.01$, and $K_d = 0.01$. In Complex Synthetic Environment and CSTR Simulator experiments, the same parameters were used as a baseline for comparison with the CNO algorithm.

N CSTR Simulator

The data generation environment used in this study is a simulator for a continuous stirred tank reactor (CSTR), a non-linear system widely known in the field of chemical process control. This simulator is implemented using the Python Control Systems Library [Fuller *et al.*, 2021], based on the model introduced in the literature [Uppal *et al.*, 1974; Marlin and Marlin, 1995; Sinha and Mishra, 2018].

N.1 System Dynamics

The system is described by the following set of differential equations.

$$\dot{C}_A = \frac{F}{V} (C_{A,\text{in}} - C_A) - k_T C_A = f_{\dot{C}_A}(C_A, T) \quad (16)$$

$$\begin{aligned} \dot{T} &= \frac{F}{V} (T_{\text{in}} - T) + \frac{k_T (-\Delta H) C_A}{\rho c_p} - \frac{AU}{\rho V c_p} (T - T_{\text{jacket}}) \\ &= f_{\dot{T}}(C_A, T, T_{\text{jacket}}) \end{aligned} \quad (17)$$

$$k_T = k_0 \exp \left(\frac{-E}{RT} \right) \quad (18)$$

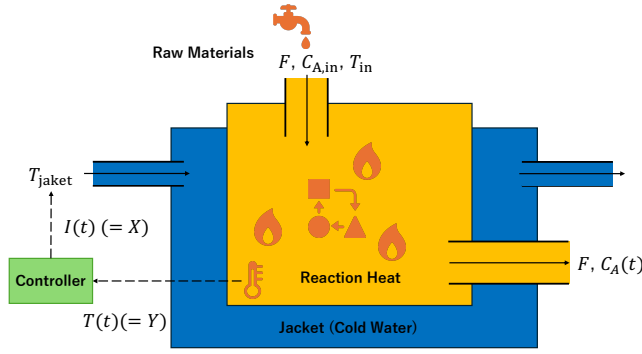


Figure 1: CSTR System Diagram.

Equation (16): Governing Equations (Mass Balance)

- **Term 1:** Change in concentration due to inflow/outflow.
 - T : Temperature of the reactor (variable)
 - F : Inflow/outflow (constant)
 - V : Reactor volume (constant)
 - $C_{A,in}$: Concentration of reactant A in the inflow (constant)
 - C_A : Concentration of reactant A in the reactor (variable)
- **Term 2:** Decrease in concentration due to chemical reaction.
 - k_T : Reaction rate constant (depends on temperature)

Equation (17): Energy Balance

This describes the time evolution of the reactor temperature (T).

- **Term 1:** Heat exchange due to inflow/outflow.
 - T_{in} : Temperature of the inflow (constant)
- **Term 2:** Heat generation due to chemical reaction.
 - $(-\Delta H)$: Heat of reaction (positive for an exothermic reaction) (constant)
 - ρ : Density (constant)
 - c_p : Specific heat capacity (constant)
- **Term 3:** Heat removal by the cooling jacket.
 - U : Overall heat transfer coefficient (constant)
 - A : Heat transfer area (constant)
 - T_{jaket} : Cooling jacket temperature (variable)

Equation (18): Arrhenius Equation

This is the source of the system's nonlinearity.

- k_0 : Frequency factor (constant)
- E : Activation energy (constant)
- R : Gas constant (constant)

PID Controller (19)

Moreover, we assume that PID control is already working. In this case, we define:

- **Manipulated variable:** $I = T_{jaket}$
- **Variable to be optimized:** T
- **Setpoint:** T_{ref} (desired temperature)

$$I = K_P(T_{ref} - T) + K_I \int_0^t (T_{ref} - T)dt + K_D \frac{d(T_{ref} - T)}{dt} \quad (19)$$

N.2 Translation into Structural Equations Modeling

The time derivative of the control input (19) is:

$$\dot{I} = -K_I(T - T_{ref}) - K_P\dot{T} - K_D\ddot{T} = f_I(T, \dot{T}, \ddot{T})$$

From Chain Rule,

$$\dot{T} = f_T(C_A, T, T_{jaket}) = f_T(C_A, T, I) \quad (20)$$

$$\therefore \ddot{T} = \frac{\partial f_T}{\partial C_A} \dot{C}_A + \frac{\partial f_T}{\partial T} \dot{T} + \frac{\partial f_T}{\partial I} \dot{I} \quad (21)$$

Circular references is created in $\ddot{T}$ and $\dot{I}$. We solve it from the original formula.

$$\dot{I} = -K_I T - K_P \dot{T} - K_D \ddot{T}$$

$$\ddot{T} = \frac{\partial f_T}{\partial C_A} \dot{C}_A + \frac{\partial f_T}{\partial T} \dot{T} + \frac{\partial f_T}{\partial I} \dot{I} \quad (f_T = f_T(C_A, T, I))$$

Both are linear, so you can simply substitute and solve.

$$\therefore \ddot{T} = g_{\ddot{T}}(C_A, T, I, \dot{C}_A, \dot{T})$$

$$\therefore \dot{I} = g_I(C_A, T, I, \dot{C}_A, \dot{T})$$

It should be possible to express in the above form. It is no longer necessary to consider $\ddot{T}^{(t)}$ as the smallest element.

Discretize in the time direction. Using the differential equation, we obtain the following.

$$\dot{C}_A^{(t)} = f_{\dot{C}_A}(C_A^{(t)}, T^{(t)}) \quad (22)$$

$$C_A^{(t+1)} \simeq C_A^{(t)} + \dot{C}_A^{(t)} \Delta t \quad (23)$$

$$\dot{T}^{(t)} = f_T(C_A^{(t)}, T^{(t)}, I^{(t)}) \quad (24)$$

$$T^{(t+1)} \simeq T^{(t)} + \dot{T}^{(t)} \Delta t \quad (25)$$

$$\dot{I}^{(t)} = g_I(C_A^{(t)}, T^{(t)}, I^{(t)}, \dot{C}_A^{(t)}, \dot{T}^{(t)}) \quad (26)$$

$$I^{(t+1)} \simeq I^{(t)} + \dot{I}^{(t)} \Delta t \quad (27)$$

Delete intermediate variables by substitution.

$$C_A^{(t+1)} \simeq C_A^{(t)} + f_{\dot{C}_A}(C_A^{(t)}, T^{(t)}) \Delta t \quad (28)$$

$$T^{(t+1)} \simeq T^{(t)} + f_T(C_A^{(t)}, T^{(t)}, I^{(t)}) \Delta t \quad (29)$$

$$I^{(t+1)} \simeq I^{(t)} + g_I(C_A^{(t)}, T^{(t)}, I^{(t)}, f_{\dot{C}_A}(C_A^{(t)}, T^{(t)}), / f_{\dot{T}^{(t)}}(C_A^{(t)}, T^{(t)}, I^{(t)})) \Delta t \quad (30)$$

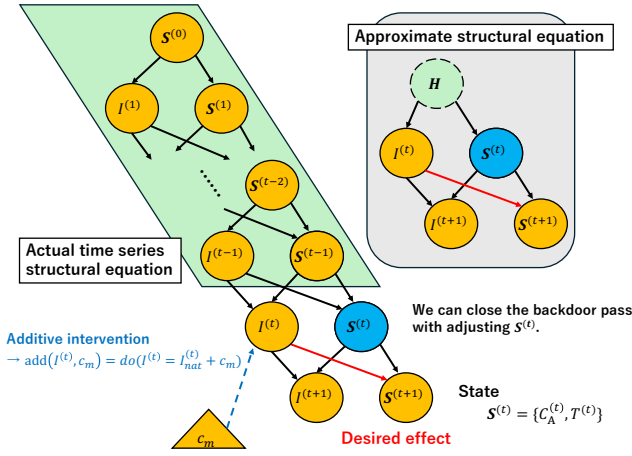


Figure 2: Structural Equation Model of CSTR.

Structural Equation Model

From the above, a simplified structural equation model is as follows.

$$V_0 = \mathbf{H} = U_0 \quad (31)$$

$$V_1 = C_A^{(t)} = f_1(\mathbf{H}, U_1) \quad (32)$$

$$V_2 = T^{(t)} = f_2(\mathbf{H}, U_2) \quad (33)$$

$$V_3 = I^{(t)} = f_3(\mathbf{H}, U_3) \quad (34)$$

$$V_4 = C_A^{(t+1)} = f_4(C_A^{(t)}, T^{(t)}, U_4) \quad (35)$$

$$V_5 = T^{(t+1)} = f_5(C_A^{(t)}, T^{(t)}, I^{(t)}, U_5) \quad (36)$$

$$V_6 = I^{(t+1)} = f_6(C_A^{(t)}, T^{(t)}, I^{(t)}, U_6) \quad (37)$$

$\mathbf{H}$ is a vector summarizing past history. $\mathbf{H}$ is the history up to $t-2$, so it is originally a time-varying (i.e., non-stationary) random variable $\mathbf{H}^{(t-2)}$, but it is approximated by assuming that it follows the same stationary distribution. (This approximation is acceptable if we consider a sufficiently long finite time-window.)

↓

$$V_0 = U_0 \quad (38)$$

$$V_1 = f_1(V_0, U_1) \quad (39)$$

$$V_2 = f_2(V_0, U_2) \quad (40)$$

$$V_3 = f_3(V_0, U_3) \quad (41)$$

$$V_4 = f_4(V_1, V_2, U_4) \quad (42)$$

$$V_5 = f_5(V_1, V_2, V_3, U_5) \quad (43)$$

$$V_6 = f_6(V_1, V_2, V_3, U_6) \quad (44)$$

Although V_0 cannot be observed directly, it is sufficient to observe $\mathbf{S}^{(t-1)} = (V_1, V_2)$ when executing the algorithm, as this allows for adjustment of causal effects.

N.3 Simulation (Data Generation)

In this section, we describe the simulation setup used to evaluate the proposed method.

Simulation Parameters

Simulation parameters are set as follows, based on the CSTR model described above. The parameters are defined in the 'CSTRSimulator' class in the file 'mymodules/environments/cstr_simulator.py'.

```
self.params["CA_in"] = 8.24
# [kmol/m^3]
self.params["cp"] = 3.14e3 # [J/kg-K]
self.params["E"] = 69.8e6 # [J/kmol]
self.params["R"] = 8.314e3 # [J/kmol-K]
self.params["k0"] = 1.18e9 # [1/min]
self.params["k0_init"] = self.params["k0"]
self.params["delta_H"] = -69.8e6
# [J/kmol]
self.params["F"] = 0.0524 # [m^3/min]
self.params["T_in"] = 294.4 # [K]
self.params["T_in_mean"] = 294.4
self.params["A"] = 12.71 # [m^2],
self.params["U"] = 5.11e4
# [J/min-m^2-K]
self.params["V"] = 3.79
# [m^3], 1m^3=1000l
self.params["rho"] = (
    800.9 # [kg/m^3]
)
self.params["rhoj"] = 997.9 # [kg/m^3]
```

Adding Noise and Drift

In the CSTR simulator, we introduce noise and aging trend to the system to simulate real-world conditions.

- **Noise:** We add noise to the temperature of the inflow T_{in} , to simulate effects from outside temperature, and the target variable T to simulate measurement noise. The noises are generated using a normal distribution with a mean of 0 and a standard deviation of 0.1.
- **Drift:** To simulate aging effects of catalyst, we introduce nonlinear drift to frequency factor k_0 as follows:

$$k_0(t) = k_{0, \text{init}} \cdot \exp(-10^{-5} \cdot t) \quad (45)$$

O Result Validation & Interpretation

- **Statistical Tests:** We conducted Wilcoxon signed-rank tests ($\alpha = 0.05$, Holm-Bonferroni correction) over 100 trials. In the Complex Env, CNO significantly outperformed all baselines ($p_{\text{adj}} < 0.001$, RBC > 0.85 , win count $\geq 84/100$), achieving an Improvement Ratio median of 0.791 compared to BO (1.005), SAC (0.996), and PID (0.996). In the CSTR Simulator, CNO significantly outperformed SAC and BO ($p_{\text{adj}} < 0.001$, RBC > 0.65 , win count > 72). While PID statistically beat CNO in CSTR ($p_{\text{adj}} < 0.001$, RBC = -0.62), the practical difference was marginal (Ratio median: PID 0.43 vs. CNO 0.45 compared to BO's 0.50 and SAC's 0.53).
- **Interpreting Results of Table 1:** CNO achieves the best balance between objective value improvement and robustness. While the CSTR environment (a standard dynamical system) favors PID control, CNO excels in com-

plex environments that follow general SCMs. In safety-critical real world applications, this robust stability is vital. Similar to [Bareinboim et al., 2015], this research demonstrates advantage of confounding adjustment as significant contribution.

P Related Work & Novelty

- **Related Work in Calibration:** There are related works regarding "calibration" settings in various fields. In control engineering, **bias compensators** and **cascade control** share similar concepts; they add an outer control loop to an existing inner control system to minimize the bias from a target value. While data-driven methods exist in this domain, they typically require dedicated pre-training periods or artificial excitation noise to mitigate confounding, and often assume global linearity. **Safe Opt./BO/RL** manage safety via constrained optimization or trust regions (i.e., adaptive intervention ranges). These safe extensions, as well as common RL methods, fundamentally suffer from the exploration-exploitation trade-off due to their black-box multivariate modeling with broad exploration range. Furthermore, they often rely on global surrogate models, which can be unstable under distribution shifts and have computational scalability issues. **Residual Reinforcement Learning** shares our concept of applying additive corrections to an easily obtainable base controller, e.g., behavioral cloning. However, because it must implicitly learn causal models to remove confounding bias through trial-and-error rewards, it still fundamentally suffers from poor exploration efficiency. Note that, as **general challenges of calibration settings**, naive univariate BO/RL models fail in calibration by assuming static relations ($Y = \hat{f}(X)$) rather than true additive ones ($Y = f(X + X_{\text{nat}})$). While contextual models (e.g., Contextual BO) address this via $Y = f(X, X_{\text{nat}})$, they require expensive and unstable global multivariate modeling. Crucially, lacking explicit causal structures, their optimizations remain fundamentally unjustified: generally $\max_X L(Y|X, X_{\text{nat}}) \neq \max_X L(Y|\text{do}(X), X_{\text{nat}})$.
- **CNO's Novelty & Justification:** CNO's distinct novelty lies in effectively exploiting the "semi-optimized state" as an inductive bias. It can justify a local convex approximation, enabling efficient online optimization via local linear modeling across nonlinear classes. It also allows inherent fluctuations as a cost-free resource for causal identification via back-door adjustment. CNO reduces modeling costs and separates the effects of Z from X through an explicit local linear causal model, enabling ideal utilization of the semi-optimized inputs Z .

Q Practical Verification of Assumptions

Verification of the Local Neighborhood Assumption

Strictly verifying whether the system is "sufficiently close" to the optimal state (as required by Assumption 1) without access to an oracle is theoretically challenging. However, to

ensure the stability and reliability of the CNO algorithm in practice, we rely on two empirical approaches[cite: 938]:

1. **Online Diagnostics:** By continuously monitoring the fluctuations of the (log-squared) residuals $\ln((r^{(t)})^2)$, we can empirically detect potential violations of the local linearity assumption during online operation[cite: 939].
2. **Algorithmic Safeguards:** As detailed in Proposition 3 (Appendix F), high uncertainty in the causal effect estimation inherently acts as a damping factor[cite: 939]. This statistically reduces the Newton step size, effectively preventing divergence even if the system temporarily deviates from the assumed local neighborhood[cite: 939].

References

- [Fujimoto et al., 2018] Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*, pages 1587–1596. PMLR, 2018.
- [Fuller et al., 2021] Sawyer Fuller, Ben Greiner, Jason Moore, Richard Murray, René van Paassen, and Rory Yorke. The python control systems library (python-control). In *60th IEEE Conference on Decision and Control (CDC)*, pages 4875–4881. IEEE, 2021.
- [Haarnoja et al., 2018] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.
- [Ljung, 1998] Lennart Ljung. System identification. In *Signal analysis and prediction*, pages 163–173. Springer, 1998.
- [Marlin and Marlin, 1995] Thomas E Marlin and TE Marlin. *Process control: designing processes and control systems for dynamic performance*, volume 2. McGraw-Hill New York, 1995.
- [Pearl et al., 2016] Judea Pearl, Madelyn Glymour, and Nicholas P Jewell. *Causal inference in statistics: A primer*. John Wiley & Sons, 2016.
- [Pearl, 2009] Judea Pearl. *Causality*. Cambridge university press, 2009.
- [Sinha and Mishra, 2018] Abhinav Sinha and Rajiv Kumar Mishra. Control of a nonlinear continuous stirred tank reactor via event triggered sliding modes. *Chemical Engineering Science*, 187:52–59, 2018.
- [Uppal et al., 1974] Ashok Uppal, Willis Harmon Ray, and Aubrey B Poore. On the dynamic behavior of continuous stirred tank reactors. *Chemical Engineering Science*, 29(4):967–985, 1974.